CarHunter — Master Build Spec (CEO Operating System for Vehicle Arbitrage)

CarHunter documentation · source: CARHUNTER_MASTER_SPEC.md · generated June 2026

CarHunter — Master Build Spec (CEO Operating System for Vehicle Arbitrage)

> **For the implementing coding agent.** This is the authoritative spec. CarHunter already exists as a Convex + React app with a live KSL scrape, a scoring engine, and a working Laser Appraiser browser bridge. Your job is to extend it into a CEO-level operating system. Reuse what exists (named below); build the rest. **AI gathers, normalizes, enriches, scores, summarizes, routes. Humans negotiate, inspect, approve, and buy. Every decision is logged and explainable.**

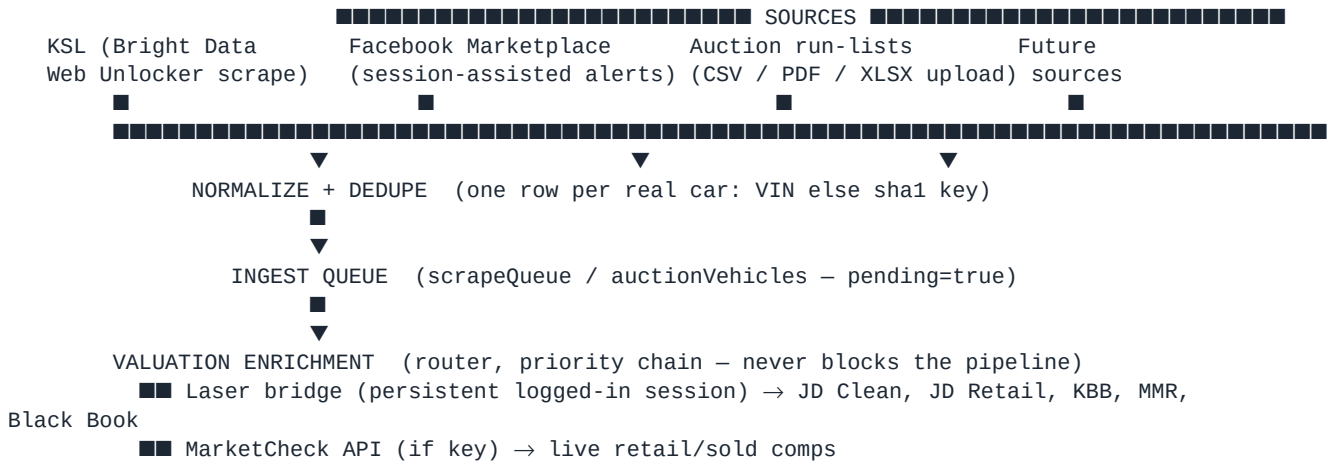
1. Product Vision (plain English)

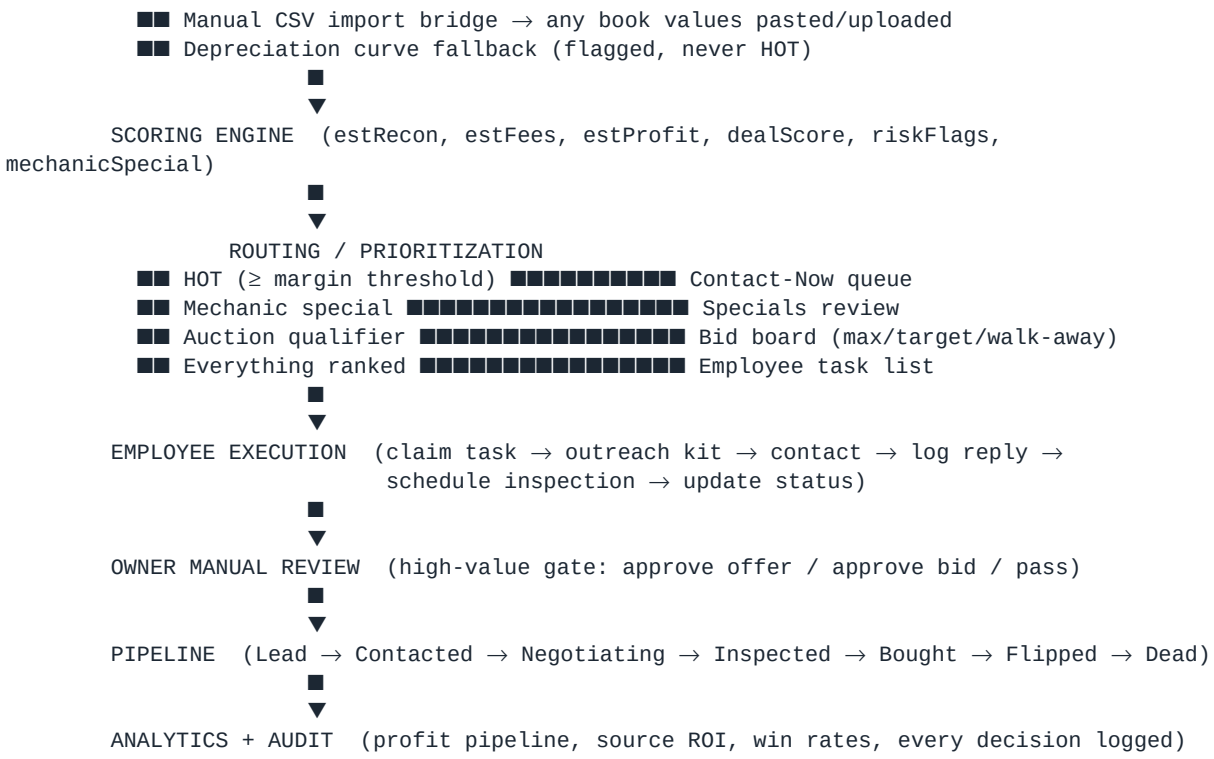
CarHunter is the **data center and decision engine** for a wholesale car-flipping operation. Every new private-party listing, every marketplace post, and every auction run-list vehicle is **instantly pulled in, valued against every book (JD Clean Trade, JD Full Retail, KBB Lending, MMR), recon- and fee-adjusted, scored, risk-flagged, and queued** for a human.

- The **owner (CEO)** opens one dashboard and sees: today's best deals, the contact-now queue, auction opportunities, employee progress, pending decisions, and the profit pipeline — and spends time only on **final high-value judgment**.
- An **employee** opens a clean, ranked **task list**: who to call, what to offer, what to ask, what to watch for — executes, logs the reply, schedules inspections, updates status. No guesswork, no redundant searching.
- The business gets **first dibs** on profitable cars all day, every day, because the machine never stops scanning and pre-packaging opportunities.

The product is **not** an auto-buyer or an auto-spammer. It is an always-on analyst + dispatcher. Humans remain the final manual decision makers.

2. End-to-End Workflow





Everything between SOURCES and OWNER REVIEW is **AI/automation**. Everything from EMPLOYEE EXECUTION onward is **human-driven with AI assistance**.

---

3. Data Model Changes (Convex `schema.ts`)

Keep & extend existing tables: `listings`, `scrapeQueue`, `comps`, `partsCosts`, `searches`, `pipeline`, `settings`, `scanState`.

3.1 `listings` (extend)

Add fields (replace the Carbly-specific naming with source-agnostic):

```
sourceType:        "private" | "auction" | "marketplace"
source:            "ksl" | "facebook" | "auction:<house>" | ...
# valuation block (all optional, populated by the router):
jdCleanTrade, jdFullRetail, kbbLending, mmrBase, blackBook: number
askingPrice:       number        # private party ask, or lane estimate for auction
estRecon:          number
estFees:           number
estProfit:         number        # see §9
dealScore:         number        # 0–100
buyTarget, walkAway, maxBid: number   # §9 / §7
riskFlags:         string[]      # e.g. ["branded_title","high_miles","thin_comps",
"price_outlier","stale_value"]
mechanicSpecial:   boolean
mechanicReason:    string | null # which keyword/rule fired
valuationSource:   string        # "laser" | "marketcheck" | "manual" | "curve" | mixed
valuationAt:       number        # ms timestamp of newest book value
assignedTo:        Id<"users"> | null
reviewState:       "auto" | "queued" | "owner_review" | "approved" | "passed"
```

3.2 NEW tables

```
valuations         # immutable per-source history (auditable)
  listingId, vin, source ("laser_jd_clean"|"laser_jd_retail"|"laser_kbb"|"laser_mmr"|
```

```
  "marketcheck"|"manual_csv"|"curve"), value, mileage, capturedAt, byUser?, raw (string)

tasks              # the employee work queue
  listingId, type ("contact"|"followup"|"inspect"|"bid_review"|"owner_decision"),
  status ("open"|"claimed"|"in_progress"|"waiting_reply"|"done"|"dropped"),
  assignee (Id<"users">|null), priority (number), dueAt, createdAt,
  outcome ("contacted"|"no_answer"|"negotiating"|"scheduled"|"bought"|"dead"|null),
  notes, log: [{ at, byUser, event, detail }]

outreach           # the per-lead outreach kit + thread
  listingId, channel ("manual"|"sms"|"email"), status (
"draft"|"approved"|"sent"|"replied"|"closed"),
  suggestedMessage, offerLow, offerTarget, offerWalkAway,
  questions: string[], redFlags: string[],
  thread: [{ at, dir ("out"|"in"), byUser?, body }]

auctionRuns        # an uploaded run-list
  house, laneDate, fileName, fileStorageId, uploadedBy, rowCount,
  status ("uploaded"|"parsing"|"enriching"|"ready"|"error"), error?

auctionVehicles    # one row per run-list vehicle (mirrors listings valuation block)
  runId, vin, year, make, model, trim, mileage, lane, announcedCondition,
  laneEstimate, <valuation block from 3.1>, maxBid, buyTarget, walkAway,
  recommendation ("bid"|"watch"|"pass"), decision ("bid"|"pass"|null), decidedBy?

users              # auth + roles
  email, name, role ("owner"|"manager"|"employee"), active

auditLog           # explainability — every state change & decision
  at, byUser ("system"|Id<"users">), entity ("listing"|"task"|"auction"|"outreach"),
  entityId, action, before?, after?, reason?

followups          # reminders
  listingId|taskId, dueAt, kind, note, done
```

3.3 Decommission

- `carbly*` fields on `listings` → migrate into the generic valuation block + `valuations` rows (source = "laser_*"). Keep a read shim during migration.

---

4. Backend Architecture (Convex)

Reuse the existing Convex deployment. Organize functions by domain:

- **Ingestion** — `http.ts` (`POST /ingest` for scrapers & the Laser bridge, `POST /laser/values`, `GET /laser/pending`, all secret-gated + CORS), `auction.ts` (CSV/PDF/XLSX parse actions), `listings.upsertFromScrape`, `lib/dedupe.ts`, `lib/listingValidator.ts`.
- **Valuation router** — NEW `valuation.ts`: `enqueueForValuation`, `applyValuation` (writes `valuations` + the listing block), `needsValuation` query, and a **priority chain** (Laser → MarketCheck → manual → curve). Replaces the Carbly-coupled path. `lib/marketcheck.ts` stays as one provider.
- **Scoring** — `scoring.ts` + `lib/scoreMath.ts` + `lib/reconRules.ts` (extend; see §9). Triggered after every valuation write.
- **Routing/queue** — NEW `routing.ts`: turns scored listings into `tasks`, contact-now entries, and bid-board rows; sets `reviewState`.
- **Task engine** — NEW `tasks.ts`: claim/assign/update/complete, SLA timers.
- **Outreach engine** — NEW `outreach.ts`: build kit (templated), approve, send (via approved channel), log replies.

- **Auction** — NEW `auction.ts`: run upload → parse → enrich → bid math → board.
- **Scrapers** (external) — `scrapers/ksl_unlocker.py` + `ksl_detail.py` via Bright Data; future `facebook.py` (session-assisted); auction parser.
- **Bridge** — `laser.ts` (the browser bridge endpoints), generalized to capture **all** Laser books (JD clean, JD retail, KBB, MMR, Black Book) per VIN.
- **Crons** — `crons.ts` (§12). **Notifications** — `alerts.ts`/`notifications.ts`.
- **Auth** — Convex auth + `users` roles; every mutation checks role.
- **Audit** — `auditLog` written by a shared helper on every state transition.

---

5. Frontend Screens (React + Vite + Tailwind)

Role-aware (owner vs employee). Screens:

1. **CEO Dashboard** (owner) — §8 widgets.
2. **Deal Feed** — ranked listings; filters (source, title, price, score, HOT, mechanic special); each card shows the full valuation strip (§3.1).
3. **Contact-Now Queue** — HOT + mechanic specials awaiting first contact.
4. **Lead Detail** — full valuation breakdown, comps, recon line items, risk flags, **outreach kit** (message + offer range + questions + red flags), thread log, status controls.
5. **Auction Board** — uploaded runs; per-vehicle enriched values + max/target/ walk-away + recommendation; bulk "approve to bid" + export bid sheet.
6. **Run-List Upload** — drag/drop CSV/PDF/XLSX, column-mapping UI, preview.
7. **Valuation Import** — paste/upload book values (manual bridge fallback).
8. **Employee Task List** — claimable, ranked, with one-click status updates.
9. **Pipeline Board** — Lead → … → Flipped (drag between stages).
10. **Owner Review** — pending high-value approvals (offers/bids) with the AI rationale; approve/pass with reason.
11. **Settings** — buy-box thresholds, fees, recon defaults, sources, users, valuation source priority, secrets status (not values).
12. **Audit / Decision Log** — searchable, exportable.

---

6. Employee Task System

- **AI creates tasks** from scored listings (`routing.ts`): every HOT or qualified lead → a `contact` task with priority = dealScore-weighted urgency (freshness + profit + competition risk). Mechanic specials always become tasks regardless of computed profit.
- **One queue, claimable.** Employee sees a ranked list; claims the top task; the lead detail opens with the outreach kit pre-built.
- **Simple status verbs:** Contacted · No answer · Negotiating · Scheduled inspection · Bought · Dead. Each writes to `tasks.log` + `auditLog`.
- **SLA + reminders.** Fresh HOT leads get a tight due time; overdue tasks surface to managers. Follow-ups auto-generate (`followups`).

- **Escalation.** When a deal exceeds an owner-review threshold (e.g. profit ≥ $X or spend ≥ $Y), the task becomes an `owner_decision` and routes to Owner Review.
- Employees can **never** approve a purchase/bid above threshold — only execute and recommend.

---

7. Auction Run-List Upload System

1. **Upload** CSV / PDF / XLSX (`Run-List Upload` screen → Convex file storage → `auctionRuns` row, status "parsing").
2. **Parse** (`auction.ts` action):
3. CSV/XLSX: header detection + **column-mapping UI** (VIN, year/make/model, mileage, lane, run #, announcements). Save mapping per house for reuse.
4. PDF: text extract; OCR fallback for scanned sheets; VIN regex (`[A-HJ-NPR-Z0-9]{17}`) as the anchor; fuzzy column inference.
5. One `auctionVehicles` row per parsed vehicle; dedupe by VIN.
6. **Enrich** every vehicle through the **same valuation router** (§8) — JD Clean, JD Retail, KBB Lending, **MMR (base + lane-adjusted)**.
7. **Bid math** (§9): `maxBid`, `buyTarget`, `walkAway`, plus a `recommendation` (bid / watch / pass) and risk flags (announced damage, title, high miles).
8. **Review board**: sortable by projected profit; owner/manager bulk-approves a bid list; **manual review is always final before bidding**.
9. **Export** a clean bid sheet (CSV/print) the buyer takes to the lane.

---

8. Valuation Import / Enrichment System

Constraint honored: do **not** assume official JD/KBB/Laser/MMR APIs exist. Use a **priority chain** so the pipeline never blocks, with the most reliable source first and graceful fallback:

Priority	Source	Mechanism	Gives
1	**Laser Appraiser bridge**	persistent logged-in browser session (userscript) reads each VIN in-session and POSTs book values to `/laser/values`	JD Clean, JD Full Retail, KBB Lending, MMR, Black Book
2	**MarketCheck API**	official paid API when `MARKETCHECK_KEY` is set	live retail + sold comps
3	**Manual CSV import**	owner/employee uploads or pastes book values keyed by VIN	any book
4	**Depreciation curve**	offline fallback in `lib/depreciationCurve.ts`	rough resale, **flagged** `curve`**, never HOT**

Rules:

- The **browser-assisted session is the primary book-value path** (it's what works today and respects that there's no public Laser API). It runs in the owner's authenticated session; **no Laser credentials are stored server-side**.
- **Avoid fragile cookie scraping as the backbone.** Prefer: official API where available → session-assisted bridge → CSV import → curve. Cookie/session scraping is acceptable only inside the user's own authenticated session (current bridge), never as a headless credential-replay service.

- Cache book values in `valuations` with `capturedAt`; mark **stale** after a configurable window (e.g. 14 days) → re-enqueue. Surface `stale_value` risk flag.
- Every value is **provenance-tagged** (source, time, raw snippet) for audit.

---

9. Lead Scoring Formula

Extend `lib/scoreMath.ts` / `lib/reconRules.ts`. All numbers live in `settings` (RULES-protected; current defaults shown).

Book selection

```
buyBook   = max(jdCleanTrade, mmrBase)          # wholesale floor — what you can buy/own at
resaleBook= max(jdFullRetail, kbbLending)       # realistic exit
estValue  = resaleBook (private retail flip) OR buyBook (wholesale/auction flip)  #
configurable per playbook
```

Profit & numbers

```
estRecon  = base($400) + drivetrain parts+labor (reconRules) + §4 condition bumps
estFees   = settings.feesFlat ($400) + auctionFees (if auction)
estProfit = estValue − price − estRecon − estFees
buyTarget = estValue − estRecon − estFees − marginThreshold        # what to pay
walkAway  = estValue − estRecon − estFees − marginThreshold*0.6
maxBid    = buyTarget − auctionBuffer                               # auctions only
```

Deal score (0–100) — keep the §4 weights:

```
dealScore = 45·clamp(estProfit/4000,0,1)
          + 20·clamp((estValue−price)/estValue,0,1)
          + 15·freshness(daysListed)        # newer = higher
          + 10·mileageFit(mileage, band)
          + 10·titleBonus(titleStatus)      # clean 1.0 / unknown .4 / rebuilt .2 / salvage
.1
HOT      = estProfit ≥ marginThreshold ($1,500) AND valuationSource ≠ "curve"
```

Adjusters (existing): AWD/4WD +3%, ±$0.06/mi vs bucket median, salvage/rebuilt estValue = 0.70 × clean comp.

Risk flags (auto): `branded_title`, `high_miles`, `thin_comps`, `stale_value`, `mechanic_special`, `price_outlier` (price < 35% of book → likely error/scam/severe damage → flag, never auto-trust).

Mechanic-special detection (`reconRules.classifyDrivetrain`, extend keyword set): needs/bad/blown engine or motor, head gasket, rod knock, no compression, seized; needs transmission, no reverse, slips, won't shift; won't start, doesn't run, dead; salvage/rebuilt/branded; "mechanic special", "project", "as-is". Each match sets `mechanicSpecial=true` + `mechanicReason` and **always queues for manual review** regardless of computed profit (hidden-opportunity rule).

---

10. Outreach Workflow (human-sent, never spam)

- When a lead qualifies, `outreach.ts` builds a **kit** attached to the lead:
- **Suggested message** (templated by deal type: clean retail flip vs mechanic special vs auction-adjacent), polite, references the listing.
- **Offer range:** `offerWalkAway` (open low) → `offerTarget` (buyTarget).
- **Questions to ask:** title in hand? VIN confirm? mechanical issues? accident history? why selling? clean carfax? test-drive/inspection ok?
- **Red flags:** branded title, price outlier, mismatched photos, curbstoner signals.

- **Human in the loop:** the kit is a **draft**. An employee reviews → edits → **approves/sends** (manual, or via an approved SMS/email channel with rate limits). Nothing sends automatically. Every send + reply logs to `outreach.thread`.
- **No mass blasting.** One-to-one only; per-day send caps; opt-out respected; comply with platform ToS and messaging law (consent, identification).

---

11. Manual-Review Workflow

- **Two gates.** (1) Employee execution (contact, negotiate, inspect) needs no owner sign-off. (2) **Committing money** — making an offer above a threshold, or bidding — routes to **Owner Review** with the full AI rationale (valuations, comps, recon, risk flags, target/walk-away).
- Owner action = **Approve / Pass / Adjust** with a required `reason` → `auditLog`. Approved offers/bids unlock the employee to proceed.
- Auctions: the bid board requires explicit per-vehicle approval before it appears on the exported bid sheet.
- Nothing the system recommends is ever executed without a logged human approval.

---

12. Automation Schedule (`crons.ts`)

Job	Cadence	Action
KSL scrape	every 2 min	Bright Data Web Unlocker → `scrapeQueue` (rotating price/mileage/year grid + newest-first fast-lane)
Marketplace alerts (FB)	every 2–5 min	session-assisted fetch → queue (Phase 3+)
Valuation drain	continuous	router enriches pending VINs (bridge/API/CSV/curve)
Score + route	on valuation write	scoring → tasks/contact-now/bid board
Stale value sweep	daily	re-enqueue valuations older than window
Listing stale sweep	daily 9:00 UTC	mark unseen-48h listings `gone`
Follow-up reminders	every 15 min	fire due `followups`
Owner daily digest	1×/day	email/SMS: top deals, pending decisions, employee progress
Auction enrich	on upload	parse → enrich → bid math

---

13. Error Handling & Fallback Workflows

- **Valuation source down** → automatic fallback down the §8 chain; if all fail, set `reviewState="queued"` with `needs_manual_value` and surface in a "needs values" tray (never silently drop).
- **Scrape blocked (PerimeterX/403)** → retry w/ backoff via Web Unlocker; alert operator if sustained.
- **Bridge session expired** → bridge posts nothing, flags the listing, and the dashboard shows "Laser session needs refresh."
- **Run-list parse failure** → fall back to manual column mapping; bad rows go to a fixable error list, good rows proceed.
- **Idempotency** everywhere (dedupe keys, `markEnriched` only if price unchanged).

- **Dead-letter queue** for repeatedly-failing items + a retry button.
- **Cross-read / data sanity** (learned from the Laser bridge): validate that multi-source values describe the same VIN/vehicle; discard mismatches; flag `price_outlier` when price ■ book.
- Every failure is logged; nothing fails closed in a way that loses a lead.

---

14. Security / Secrets Plan

- **Secrets** live in Convex env / deploy host / untracked `.env.local` (gitignored) — **never committed**. Repo ships `.env.example` placeholders only.
- Current secrets: `BRIGHTDATA_API_TOKEN`, `INGEST_SECRET`, `MARKETCHECK_KEY` (optional), SMS/email provider keys, `CONVEX_DEPLOY_KEY`. Rotate any exposed in chat/logs.
- **No third-party valuation passwords stored server-side.** The Laser bridge runs inside the owner's own browser session; the server only receives book values via the `INGEST_SECRET`-gated endpoint.
- **Role-based access** (`users.role`): employees can't see secrets, can't approve spend, can't export full seller PII beyond what a task needs.
- **PII**: seller contact info is access-controlled and audit-logged; retention policy configurable.
- **Auditability**: `auditLog` is append-only; every decision is explainable (inputs + rule + actor).
- **Compliance**: respect each platform's ToS; messaging follows consent/ID law; prefer official APIs; session-assisted access only within the user's own account.

---

15. Phased Implementation Roadmap

Phase 1 — Reliable manual/CSV bridge (foundation).

- Generic valuation block on `listings` + `valuations` table; retire `carbly*`.
- **Valuation Import** screen (paste/upload book values by VIN) + **Run-List Upload** (CSV/XLSX parse + column mapping) → enriched, scored rows.
- Scoring engine reads the generic block; full valuation strip in the feed.
- Acceptance: upload a run-list or paste values → see JD/KBB/MMR + profit + score.

Phase 2 — Employee task dashboard.

- `users`/roles, `tasks`, `outreach`, `followups`; routing turns deals into tasks.
- Employee Task List + Lead Detail + outreach kit + pipeline board.
- Acceptance: an employee runs the whole day from the task list; statuses + logs.

Phase 3 — Browser-assisted valuation automation.

- Generalize the Laser bridge to capture all books (JD clean/retail, KBB, MMR, Black Book) per VIN; persistent session; auto-enrich the queue; staleness refresh.
- FB Marketplace session-assisted alerts into the queue.
- Acceptance: queued VINs auto-populate all book values with zero manual steps while a session is live; fallback chain proven.

Phase 4 — Full CEO command center.

- CEO Dashboard (all §8 widgets), Owner Review gate, Auction Bid Board with max/target/walk-away + export, daily digest, escalation thresholds.
- Acceptance: owner operates from one screen and only touches high-value decisions.

Phase 5 — Optimization & analytics.

- Source ROI, win rate by channel, scoring back-test/tuning, recon-accuracy learning, profit forecasting, A/B on outreach templates.
- Acceptance: dashboards quantify what's working; scoring weights tuned from realized outcomes.

---

16. Redundancies to Remove From the Current System

1. **Carbly everywhere** — dead path. Remove `carblyClient` from the live flow (keep `applyGapRule` math, rename generic); migrate `carbly*` fields → generic valuation block + `valuations`; drop the Carbly cooldown/`scanState` special-casing.
2. **Two appraisal paths racing** — the MarketCheck/curve `enrichTick` auto-drain vs the Laser bridge. Unify behind **one valuation router** (§8) with priority chain; delete the `LASER_BRIDGE`/`CARBLY_ENRICH` flag tangle in favor of a single ordered source list in `settings`.
3. **Duplicate scrape entry points** — `scanBand`/`runScan`/`scanNow` vs `scrapeTick`. Keep one scrape→queue path; `scanNow` becomes a manual trigger of it.
4. **Hardcoded** `valuationSource:"carbly"` in `dealUpsert` — make it reflect the real source.
5. **Deactivated seeds / legacy YMM buy-boxes** not in use — remove or archive.
6. **Multiple ad-hoc exports & report PDFs / plan files** in the repo root — move to `docs/` or gitignore; keep the data center as the source of truth.
7. **Notification provider sprawl** (MTA vs Twilio/Resend) — pick one per channel.

---

17. Exact Acceptance Criteria for Launch

Data & valuation

- [ ] Every new listing (private or auction) is normalized, deduped (one row per real car), and **valued within 10 minutes** of ingest, or explicitly flagged `needs_manual_value`.
- [ ] Each vehicle shows **all available books** (JD Clean, JD Retail, KBB Lending, MMR), asking/lane price, est recon, est fees, est profit, deal score, risk flags — with **provenance** (source + timestamp) on every value.
- [ ] Curve-only valuations are visibly flagged and **never** marked HOT.

Mechanic specials

- [ ] The keyword/rule set flags ≥ 95% of a labeled test set of needs-engine / blown-motor / no-reverse / won't-start / branded listings, and every flagged car appears in Specials review regardless of computed profit.

Auction

- [ ] A CSV/PDF/XLSX run-list uploads, parses (with column mapping), enriches, and produces a bid sheet with max/target/walk-away **in one pass**, manual approval required before export.

Employee delegation

- [ ] An employee can run an entire day from the task list: claim → outreach kit → contact → log reply → schedule → update status, with **no owner involvement** until an owner-review threshold trips.
- [ ] Outreach never sends without human approval; per-day caps + opt-out enforced.

Owner / CEO

- [ ] Dashboard shows today's best deals, contact-now queue, auction opportunities, employee progress, pending decisions, missed/expired deals, profit pipeline, follow-up reminders.
- [ ] Every money-committing action passes a logged owner approval with a reason.

Reliability & security

- [ ] All four valuation fallbacks tested; pipeline never blocks on a dead source.
- [ ] No secrets in the repo; `.env.example` only; roles enforced on every mutation.
- [ ] `auditLog` captures every state change and decision; exportable.
- [ ] Scrape + queue + score + route runs unattended for 72h with no manual fix.

Appendix A — What already exists (reuse, don't rebuild)

- Convex deployment `silent-leopard-39`; tables `listings`, `scrapeQueue`, `comps`, `partsCosts`, `searches`, `pipeline`, `settings`, `scanState`.
- Scoring: `lib/scoreMath.ts` (weights 45/20/15/10/10, HOT ≥ $1,500, fees $400, AWD +3%, ±$0.06/mi, salvage 0.70), `lib/reconRules.ts` (`classifyDrivetrain`, parts+labor recon), `lib/depreciationCurve.ts`, `lib/marketcheck.ts`.
- Ingest: `http.ts` `/ingest` (+ `/laser/pending`, `/laser/values`), `dedupe.ts`, `listingValidator.ts`.
- Scrape: `scrapers/ksl_unlocker.py`, `ksl_detail.py` (Bright Data Web Unlocker).
- **Laser bridge** (`convex/laser.ts` + `scripts/laser_bridge.user.js`): the session-assisted book-value path. Generalize it to all books in Phase 3.

Appendix B — Laser field map (already reverse-engineered, lock these in)

- JD/NADA **Clean Trade-in** = `wdVinPartnerData?partner=NADA` → `Comm.Trade` (Average `TrAv`, Rough `TrRf`); JD **Retail** = `Comm.Retail`; NADA loan = `Comm.Loan`.
- **KBB** = `partner=KBB` → `Whole` (wholesale), `Retail`, `Trade` grades.
- **MMR** = `partner=MMR`; **Black Book** = `partner=BB`.
- Auth = live-session `security` token + `deviceId` (browser bridge only).